

Université de Montréal

IFT3295 - TP1

Assemblage de Séquences

Par

Olivier LARUE

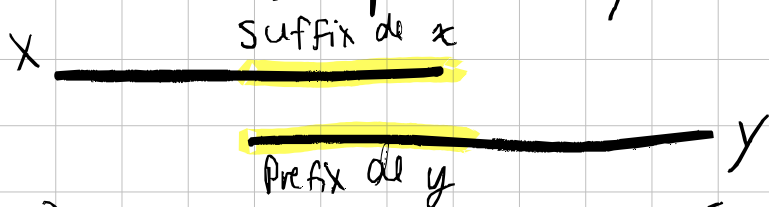
Tristhan Andres Cruz Rodriguez

9 octobre 2026

Chevauchement de séquences

Q1.

L'alignement global aligne un string x et un string y sur l'entiereté de leur longueur tandis que pour le chevauchement, on prend seulement un suffix de x et un préfix de y .



Alors, On voit aussi que le début de x n'est pas pénalisé, ainsi que la fin de y puisqu'ils ne seront pas pris en compte.

Q2.

Première ligne: $V(i, 0)$, $\forall i$

- Comme mentionné plus haut, les i premiers caractères de x peuvent se trouver avant le chevauchement, alors ils sont gratuits (comme dans local, mais seulement pour x). Alors $V(i, 0) = 0$

Première colonne: $V(0, j)$, $\forall j$:

- Le même que pour l'alignement global
 $\Rightarrow V(0, j) = j \cdot d$ où $d = \text{coût d'un INDDEL}$

Dans notre cas cela serait $d = -8 \Rightarrow V(0, j) = j \cdot -8$

Le chevauchement doit commencer avec le début de y , alors skipper les j serait comme aligner avec des —.

Chevauchement de Séquences (suite)

Q3. Récurrence:
$$S(x_i, y_i) = 4 \text{ si } x_i = y_i, -4 \text{ sinon}$$
$$\text{INDEL} = -8$$

$$V(i, j) = \max \begin{cases} V(i-1, j-1) + S(x_i, y_i) \\ V(i-1, j) + \text{INDEL} \\ V(i, j-1) + \text{INDEL} \end{cases}$$

Q4. Retrouver Alignement avec meilleur chevauchement:

On doit trouver se positionner à la dernière ligne, (la ligne) n et trouver la colonne j qui a le plus grand $D(n, j)$.
Nommons la j^* .

On commence le backtracking à partir de (n, j^*) .

Ensuite, tant que i et j sont plus grand que zéro, on vérifie les 3 cas de la récurrence pour retrouver notre chemin.

1. Si $V(i, j) = V(i-1, j-1) + S(x_i, y_i)$
 $\rightarrow$ ajouter x_i en face de y_i puis aller en $(i-1, j-1)$
2. Si $V(i, j) = V(i-1, j) - 8$
 $\rightarrow$ ajouter x_i en face d'un $-$, aller en $(i-1, j)$
3. Si $V(i, j) = V(i, j-1) - 8$
 $\rightarrow$ ajouter y_i en face d'un $-$, aller en $(i, j-1)$

On s'arrête quand $j=0$ tout préfix de y est aligné. Le préfix de x de 1 à i ne sera pas inclus dans le chevauchement.

On reverse les listes d'alignements puisqu'on

Q5: Voir implémentation dans Overlap.py

Assemblage de fragments

Q1: Voir fichier de code overlap.py:matrice-chevauchement.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
1	0	8	8	0	4	12	12	16	4	804	20	4	0	4	396	4	20	652	12	608	
2	4	0	8	12	44	8	4	416	8	12	424	16	0	1060	12	12	12	8	4	0	
3	396	12	0	24	1016	16	48	0	36	128	4	20	40	8	20	20	16	8	8	8	
4	28	8	28	0	12	640	1024	16	1068	12	12	220	16	24	12	656	152	16	8	16	
5	500	4	696	28	0	24	52	8	52	208	16	20	36	4	8	24	36	64	4	12	
6	52	4	4	16	0	0	4	4	12	28	4	16	4	0	28	8	672	16	12	20	
7	32	12	32	648	16	720	0	4	772	12	12	24	24	24	24	176	232	12	4	0	
8	8	12	644	4	540	16	8	0	8	4	944	12	4	12	0	4	8	36	4	16	
9	36	12	20	956	16	668	1044	4	0	8	4	64	12	24	20	492	196	8	4	12	
10	4	0	0	8	32	8	8	24	12	0	32	4	12	12	640	4	8	904	184	868	
11	20	20	620	4	524	12	12	1072	4	12	0	24	28	24	8	8	16	20	12	12	
12	28	28	48	900	16	460	820	0	856	4	16	0	28	24	16	996	20	16	8	4	
13	68	32	32	744	24	272	656	0	692	16	8	860	0	36	16	848	32	24	4	28	
14	16	756	12	12	52	20	8	452	12	16	492	16	8	0	4	12	20	16	8	16	
15	20	8	28	12	16	12	16	12	8	4	12	80	260	28	0	28	20	24	632	24	
16	20	16	24	1032	16	552	944	12	988	8	20	692	8	20	12	0	80	12	8	8	
17	16	4	32	16	16	32	8	12	4	24	20	12	4	28	12	8	0	16	8	20	
18	16	0	8	12	8	4	4	8	8	240	16	8	4	16	808	8	4	0	368	1004	
19	48	20	16	388	8	20	316	8	368	16	20	520	732	16	24	492	20	20	0	4	
20	8	4	0	4	0	8	4	0	0	0	4	0	64	8	908	4	16	644	428	0	
21																					

Il s'agit bien d'une matrice 20x20 représentant le résultat de ma solution à l'algorithme de prog dynamique de chevauchement. On voit qu'effectivement la matrice n'est pas symétrique et que la diagonale est égale à zéro.

Assemblage Fragments (suite)

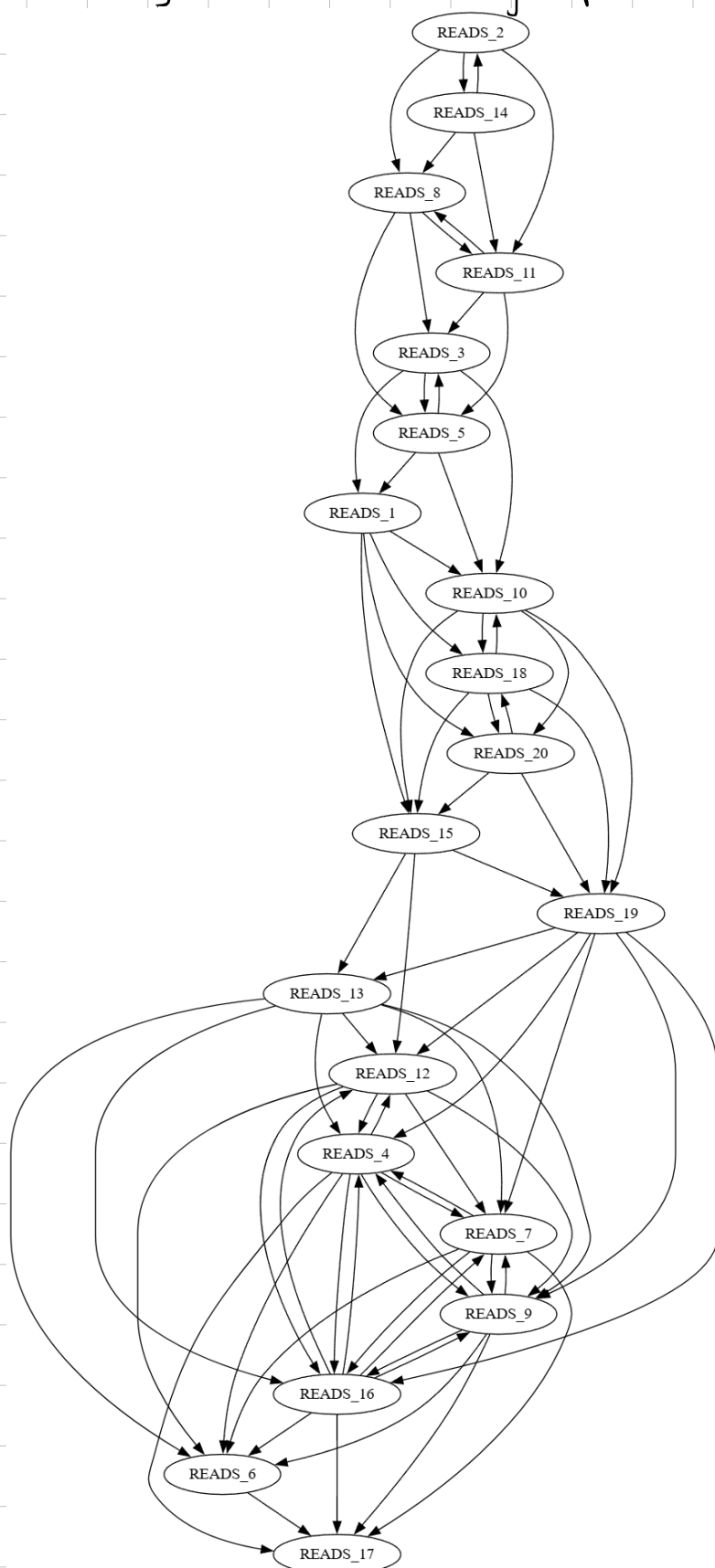
Q: Voir fichier de code assembly.py : graphe - chevauchement

→ a) filtrer par score avec seuil min. de 80.

Effet: Si l'on retire le check pour

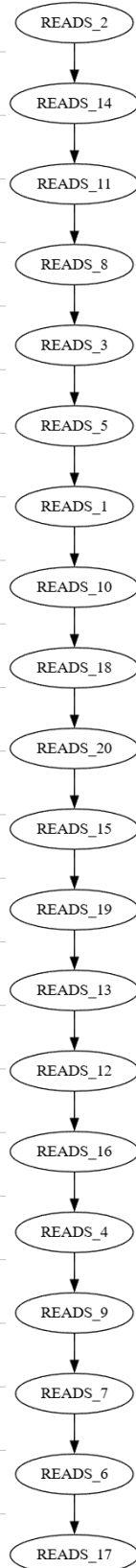
le seuil, on se retrouve avec un graphe complet, tout les noeuds sont liés par des arêtes dans les deux sens.

En mettant un seuil, on réduit le nombre d'arêtes dans le graphe. Le seuil de 80 nous permet de ne pas avoir de cycle dans le graphe.



Assemblage Fragments (suite)

5) Lors de la réduction, je retire le
edge et regarde si un chemin existe encore.
Si oui, l'edge reste retirer
Voici le résultat de la génération du
graphe réduit



c) Pour la déduction du fragment génomique, voir l'implémentation dans `assembly.py` : `sequen-finale`.

Mon intuition était la suivante :

Je me garde une liste de tout les fragments de la séquences et pour chaque read, je regarde le read précédent et fait le chevauchement maximal, me donnant l'alignement de y . Ensuite, j'ajoute seulement la partie après la longueur du chevauchement de y de Read à la position de y

ex: GTTCCA

CCA **GTG**

→ On ajoute seulement cette partie puisqu'à l'itération précédente, on a déjà ajouter "CCA".
À la fin, on colle tout les fragments ensemble.